

Turing computable function

Definition. $f \in \mathbb{N}^n \rightarrow \mathbb{N}$ is Turing computable if and only if there is a Turing machine M with the following property:

Slide 19

Starting M from its initial state with tape head on the leftmost 0 of a tape coding $[x_1, \dots, x_n]$, M halts if and only if $f(x_1, \dots, x_n) \downarrow$, and in that case the final tape codes a list (of length ≥ 1) whose first element is y where $f(x_1, \dots, x_n) = y$.

Theorem. A partial function is Turing computable if and only if it is register machine computable.

Proof (sketch). We've seen how to implement any TM by a RM. Hence

Slide 20

f TM computable implies f RM computable.

For the converse, one has to implement the computation of a RM in terms of a TM operating on a tape coding RM configurations. To do this, one has to show how to carry out the action of each type of RM instruction on the tape. It should be reasonably clear that this is possible in principle, even if the details are omitted (because they are tedious).

Notions of computability

- Church (1936): λ -calculus
- Turing (1936): Turing machines.

Slide 21

Turing showed that the two very different approaches determine the same class of computable functions. Hence:

Church-Turing Thesis. Every algorithm [in intuitive sense] can be realized as a Turing machine.

Models of computability

Church-Turing Thesis. Every algorithm can be realized as a Turing machine. Further evidence:

- Gödel and Kleene (1936): partial recursive functions

Slide 22

- Church (1936): λ -calculus
- Post (1943) and Markov (1951): canonical systems for generating the theorems of a formal system
- Lambek (1961) and Minsky (1961): register machines

- Variations on all of the above (e.g. multiple tapes, non-determinism, parallel execution...)
All determine the same collection of computable functions.

The work on Turing machines provided one path to the invention of computers, and plays a significant role in complexity theory, the classification of computational problems according to their inherent difficulty and relating these classes to each other. The work on partial recursive functions has led to a branch of mathematics called recursion theory. The work on the λ -calculus has led to a branch of computer science called functional programming.